

CENTRE FOR DEVELOPMENT OF ADVANCED COMPUTING



(AFFILIATED TO GURU GOVIND SINGH INDRAPRASTHA UNIVERSITY)

MASTER OF COMPUTER APPLICATIONS

Session : 2024-26

OBJECT ORIENTED SOFTWARE ENGINEERING LAB PRACTICAL FILE

Submitted to:

Mr. Manoj Purohit

Submitted by:

Priyanshu Tripathi

Enrolment No: 00511804424

INDEX

S.No.	LAB	Page No.
1.	Create a Use Case Diagram for a Digital Sound Recorder with the following main features: 1. The recorder stores up to 10 messages 2. Each message is max. 2 minutes long 3. The user can record message 4. Recording of a message end after 2 minutes or when the user stops recording 5. Recording destroys the original message at chosen slot 6. Sufficient level of battery is checked before recording message 7. Message of a given slot can be replayed 8. Sufficient level of battery is checked before replaying message 9. Messages can be locked/unlocked 10. Locked messages cannot be deleted or over-written by recording to the same slot User uses LCD display and buttons to interact with recorder	6
2.	Formulate use case descriptions for 5 major use cases identified in the digital sound recorder.	7
3.	As the head of Information Technology at Acme Airlines, you are tasked with building a new booking system to replace the existing system. Acme needs new system to allow employees to book ticket electronically and check the status of the booked tickets. The new system will be state of the art and will have a Windows-based desktop interface to allow employees to enter booking details and view employee details. The System will run on individual employee desktops throughout the entire company. For reasons of security and auditing, employees can access only their data. The employees would only be able to view their information, and for any changes they would have to send an e-mail to the administrator. Administrator would only have the right to make a changes in the records. For the cost being a factor Acme wants to use their old server with existing database. The system will retain information on all employees in the company. The administrator maintains employee information. He is responsible for adding new employees, deleting employees and changing all employee information such as name, address, and paycheck generation, as well as running administrative reports.	7
4.	Consider the following use-case of a travel agency. Use-Case Name: Ticket Purchasing Description: 1. The use-case begins when the customer calls the travel agency to ask it to issue a ticket that (s)he has booked. 2. The travel agency operator asks the customer to give his/her booking number. 3. The customer gives the booking number.	8

	4. The operator types in the booking number and the flight reservation system displays the details of the reservation made. 5. The operator asks the customer to confirm the details of the reservation made. 6. The customer confirms the reservation made. 7. The operator asks the customer for a credit card number. 8. The customer gives his/her credit card number. 9. The operator types in the customer's credit card number and when the system confirms that the credit card transaction has been authorised (s)he asks the system to print the tickets, the details of the flights, and an invoice. 10. When the system confirms that the requested items have been printed, the operator informs the customer that the tickets have been issued. Think of at least two alternative courses of events for this use-case. Describe the alternative courses as separate use-cases. Modify the description of the original use-case to make evident where exactly these alternative use-cases may be called and under which conditions. Create a use-case diagram to illustrate the relationships between the alternative use-cases and the use-case given.	
5.	Create a Class Diagram for a Digital Sound Recorder with the following main features: 1. The recorder stores up to 10 messages 2. Each message is max. 2 minutes long 3. The user can record message 4. Recording of a message end after 2 minutes or when the user stops recording 5. Recording destroys the original message at chosen slot 6. Sufficient Level of battery is checked before recording message 7. Message of a given slot can be replayed 8. Sufficient Level of battery is checked before replaying message 9. Messages can be locked/unlocked 10. Locked messages cannot be deleted or over-written by recording to the same slot User uses LCD display and buttons to interact with recorder	9
6.	Forward engineer the model obtained in Q5 to automatically generate equivalent Java stub Code?	9
7.	Purpose: Describe major services (functionality) provided by a hospital's reception. Summary: Hospital Management System is a large system including several subsystems modules providing variety of functions. Hospital Reception subsystem module supports some of the many job duties of hospital receptionist. Receptionist schedulespatient's appointments and admission to the hospital, collects information from patient upon patient's arrival and/or by phone. For the patient that will stay in the hospital("inpatient") she or he should have a bed allotted in a ward. Receptionist may also receive and process patient's payments, record them and provide receipts. She also is responsible to file insurance claims and generate medical reports. a. Make the use case diagram b. Provide description for at least 04 use cases c. Make the class diagram d. Generate Java stub code e. Create Object Diagram	11

8.	Create an object diagram for a Digital Sound Recorder with the following main features: 1. The recorder stores up to 10 messages 2. Each message is max. 2 minutes long 3. The user can record message 4. Recording of a message end after 2 minutes or when the user stops recording 5. Recording destroys the original message at chosen slot 6. Sufficient Level of battery is checked before recording message 7. Message of a given slot can be replayed 8. Sufficient Level of battery is checked before replaying message 9. Messages can be locked/unlocked 10. Locked messages cannot be deleted or over-written by recording to the same slot User uses LCD display and buttons to interact with recorder	19
9.	Create a package diagram for the Digital Sound Recorder in Q1.	20
10.	Create an activity diagram for the record message use case of Q1.	20
11.	Create an activity diagram for any two use cases of Hospital Reception Subsystem in Q8.	21
12.	Create a sequence diagram for the playback message use case of Q1.	22
13.	Create a sequence diagram for any two use cases of Hospital Reception Subsystem in Q7.	23
14.	Create a state diagram for any two use cases of Hospital Reception Subsystem in Q7.	24
15.	Create a state diagram for a BANK ATM machine.	25
16.	Create a communication diagram for the playback message use case of Q1.	26
17.	Create a communication diagram for any two use cases of Hospital Reception Subsystem in Q7.	26
18.	Create a component diagram for the Hospital Reception Subsystem.	27

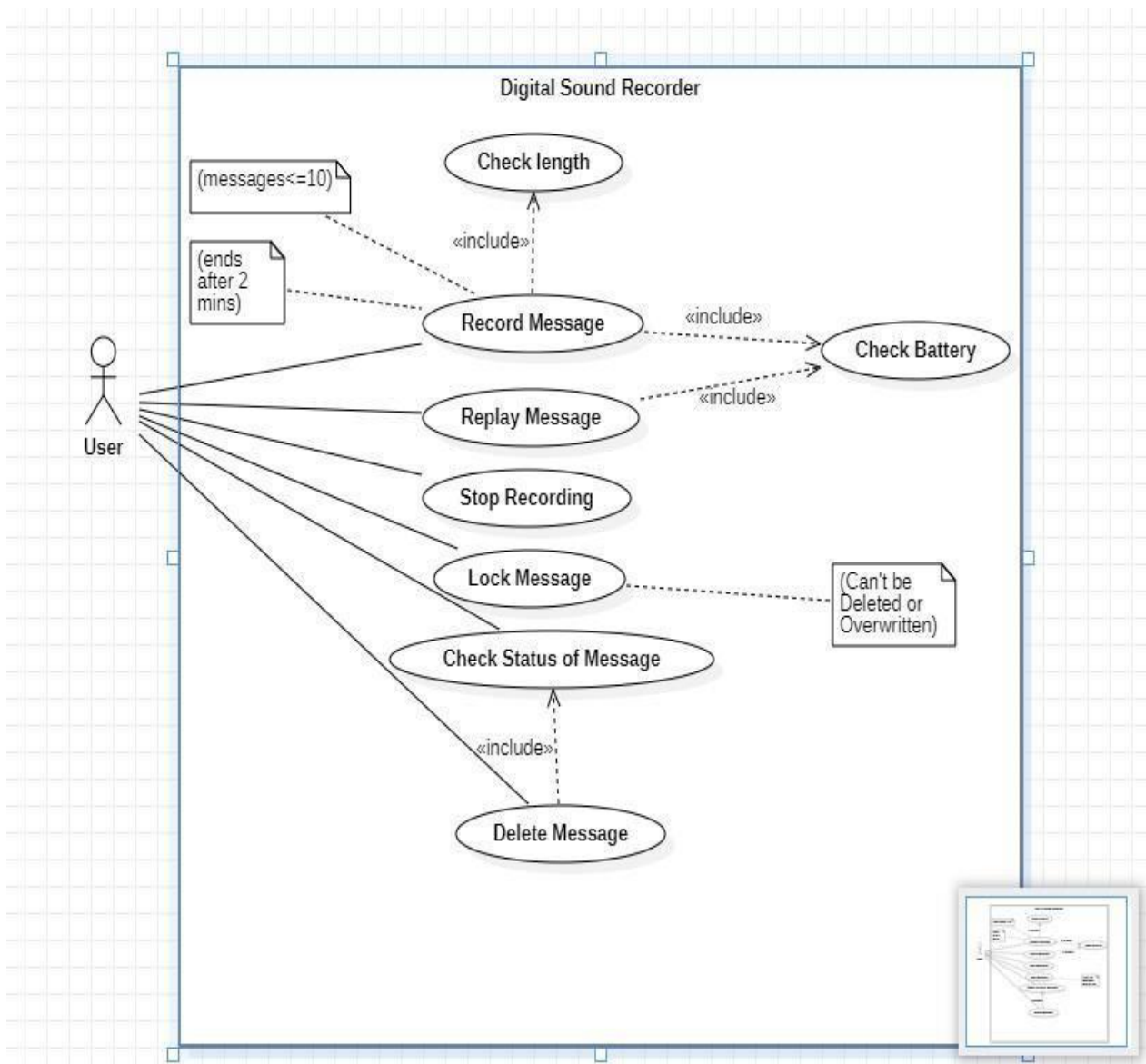
19.	Create a component diagram for ATM Machine.	28																																																
20.	Create a deployment diagram for the Hospital Reception Subsystem.	29																																																
21.	Create a deployment diagram for ATM machine.	30																																																
22.	Create a PERT Chart on PowerPoint for the following: <table><thead><tr><th>Activity</th><th>Description</th><th>Predecessor</th><th>Estimated Time (hours)</th></tr></thead><tbody><tr><td>A</td><td>Drive Home</td><td>None</td><td>0.5</td></tr><tr><td>B</td><td>Wash Clothes</td><td>A</td><td>4.0</td></tr><tr><td>C</td><td>Pack</td><td>B</td><td>4.0</td></tr><tr><td>D</td><td>Go to Bank</td><td>A</td><td>1.0</td></tr><tr><td>E</td><td>Pay Bill</td><td>D</td><td>0.5</td></tr><tr><td>F</td><td>Pack Car</td><td>C,E</td><td>0.5</td></tr><tr><td>G</td><td>Drive to Bus</td><td>F</td><td>0.5</td></tr></tbody></table>	Activity	Description	Predecessor	Estimated Time (hours)	A	Drive Home	None	0.5	B	Wash Clothes	A	4.0	C	Pack	B	4.0	D	Go to Bank	A	1.0	E	Pay Bill	D	0.5	F	Pack Car	C,E	0.5	G	Drive to Bus	F	0.5	30																
Activity	Description	Predecessor	Estimated Time (hours)																																															
A	Drive Home	None	0.5																																															
B	Wash Clothes	A	4.0																																															
C	Pack	B	4.0																																															
D	Go to Bank	A	1.0																																															
E	Pay Bill	D	0.5																																															
F	Pack Car	C,E	0.5																																															
G	Drive to Bus	F	0.5																																															
23.	Create a GANTT Chart on PowerPoint for the following: <table><thead><tr><th>Task</th><th>Prec Task</th><th>Description</th><th>Time (in hrs)</th></tr></thead><tbody><tr><td>A</td><td>None</td><td>Decide on Date for the Party</td><td>1</td></tr><tr><td>B</td><td>A</td><td>Book a bouncy Castle</td><td>1</td></tr><tr><td>C</td><td>A</td><td>Send Invitations</td><td>5</td></tr><tr><td>D</td><td>C</td><td>Receive Replies</td><td>7</td></tr><tr><td>E</td><td>D</td><td>Buy Toys and Balloons</td><td>1</td></tr><tr><td>F</td><td>D</td><td>Buy Food</td><td>3</td></tr><tr><td>G</td><td>E</td><td>Blow up Balloons</td><td>2</td></tr><tr><td>H</td><td>F</td><td>Make Food</td><td>1</td></tr><tr><td>I</td><td>H, G</td><td>Decorate</td><td>1</td></tr><tr><td>J</td><td>B</td><td>Get Bouncy Castle</td><td>1</td></tr><tr><td>K</td><td>J, I</td><td>Have Party</td><td>2</td></tr></tbody></table>	Task	Prec Task	Description	Time (in hrs)	A	None	Decide on Date for the Party	1	B	A	Book a bouncy Castle	1	C	A	Send Invitations	5	D	C	Receive Replies	7	E	D	Buy Toys and Balloons	1	F	D	Buy Food	3	G	E	Blow up Balloons	2	H	F	Make Food	1	I	H, G	Decorate	1	J	B	Get Bouncy Castle	1	K	J, I	Have Party	2	31
Task	Prec Task	Description	Time (in hrs)																																															
A	None	Decide on Date for the Party	1																																															
B	A	Book a bouncy Castle	1																																															
C	A	Send Invitations	5																																															
D	C	Receive Replies	7																																															
E	D	Buy Toys and Balloons	1																																															
F	D	Buy Food	3																																															
G	E	Blow up Balloons	2																																															
H	F	Make Food	1																																															
I	H, G	Decorate	1																																															
J	B	Get Bouncy Castle	1																																															
K	J, I	Have Party	2																																															

L	K	Cleanup	4
M	K	Send Back Bouncy Castle	1
N	M	Donate Unwanted Gifts	3

- label hours 0 to 30 across the horizontal axis
- Mark a review stage at hour 14 to monitor the progress
- Assume and illustrate that tasks A, B, C and D have been completed at hour 14. State which tasks are ahead and which tasks are behind schedule

LAB-1

Use-case Diagram:



LAB-2

Solution:

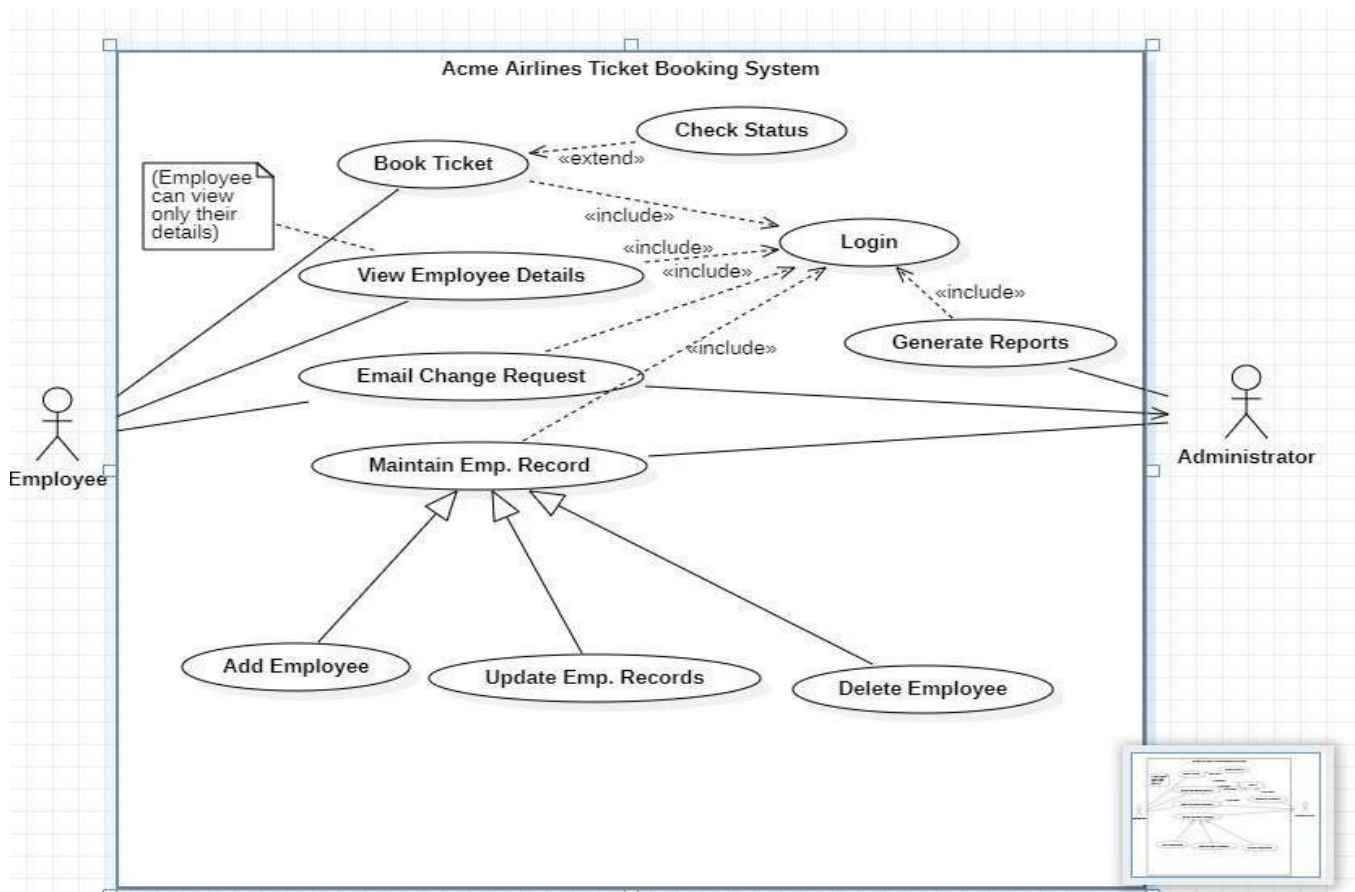
Use cases describe the functionality of the system from the user's point of view. The user may be a person or another machine. Each use case is a different way to use the system and the completion of

Use case ID	Use case	Description
001	Record the message	Record the message function allows the user to record his/her message
002	Store messages	Using this function the user can save his recordings for future references and play them later on.
003	Stop recording	This function automatically stops the recording after 2 min and this is managed by the administrator.
004	Lock Message	This function allows users to lock specific audio messages for enhanced privacy and security. By locking a message, only authorized individuals with the correct credentials or access rights can listen to or modify the locked message.
005	Replay Messages	The user also has the function to replay the recording if there is sufficient battery level left.

each use case produces a different result. In our system there are six different use cases.

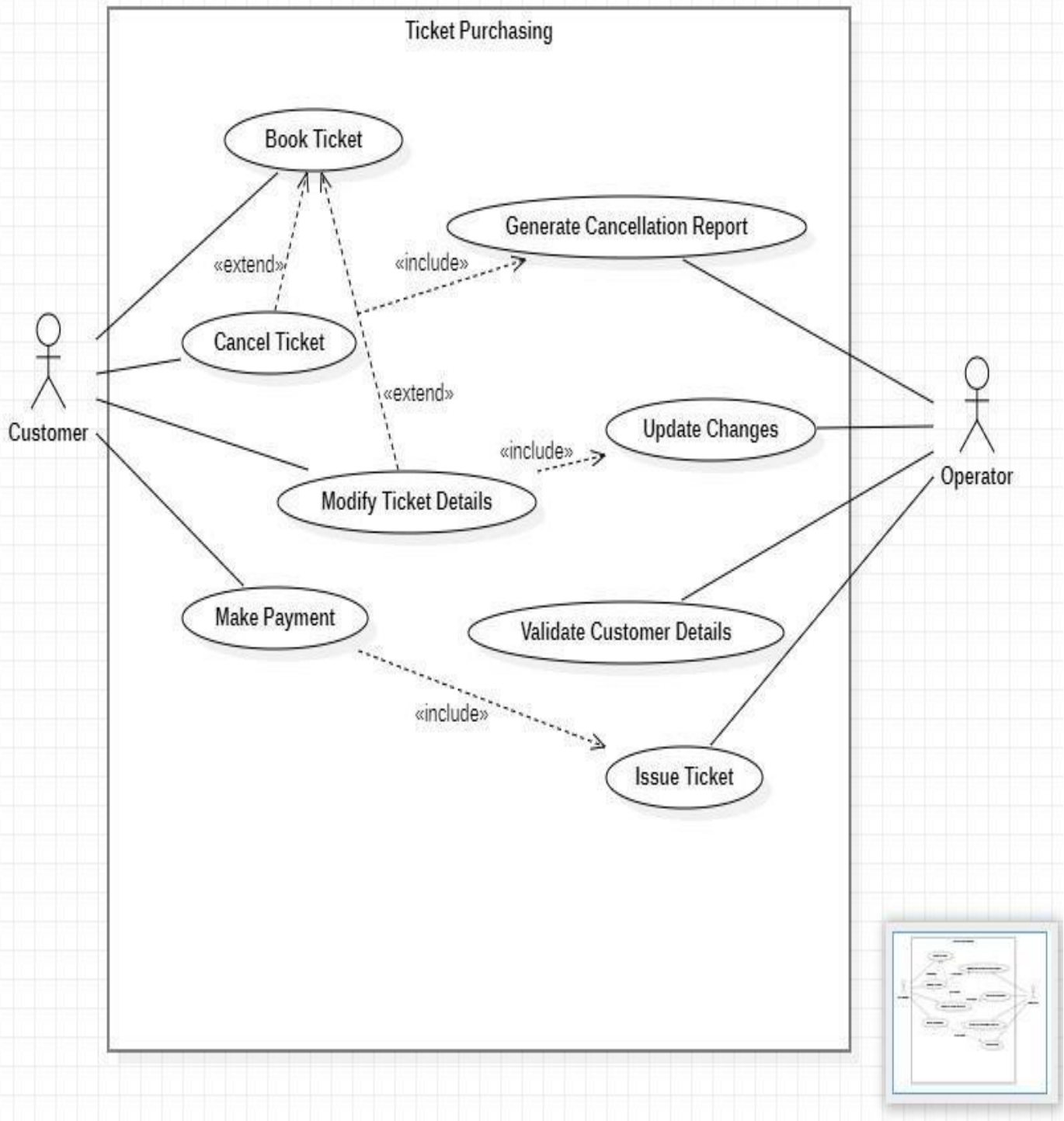
LAB-3

Use-case Diagram:



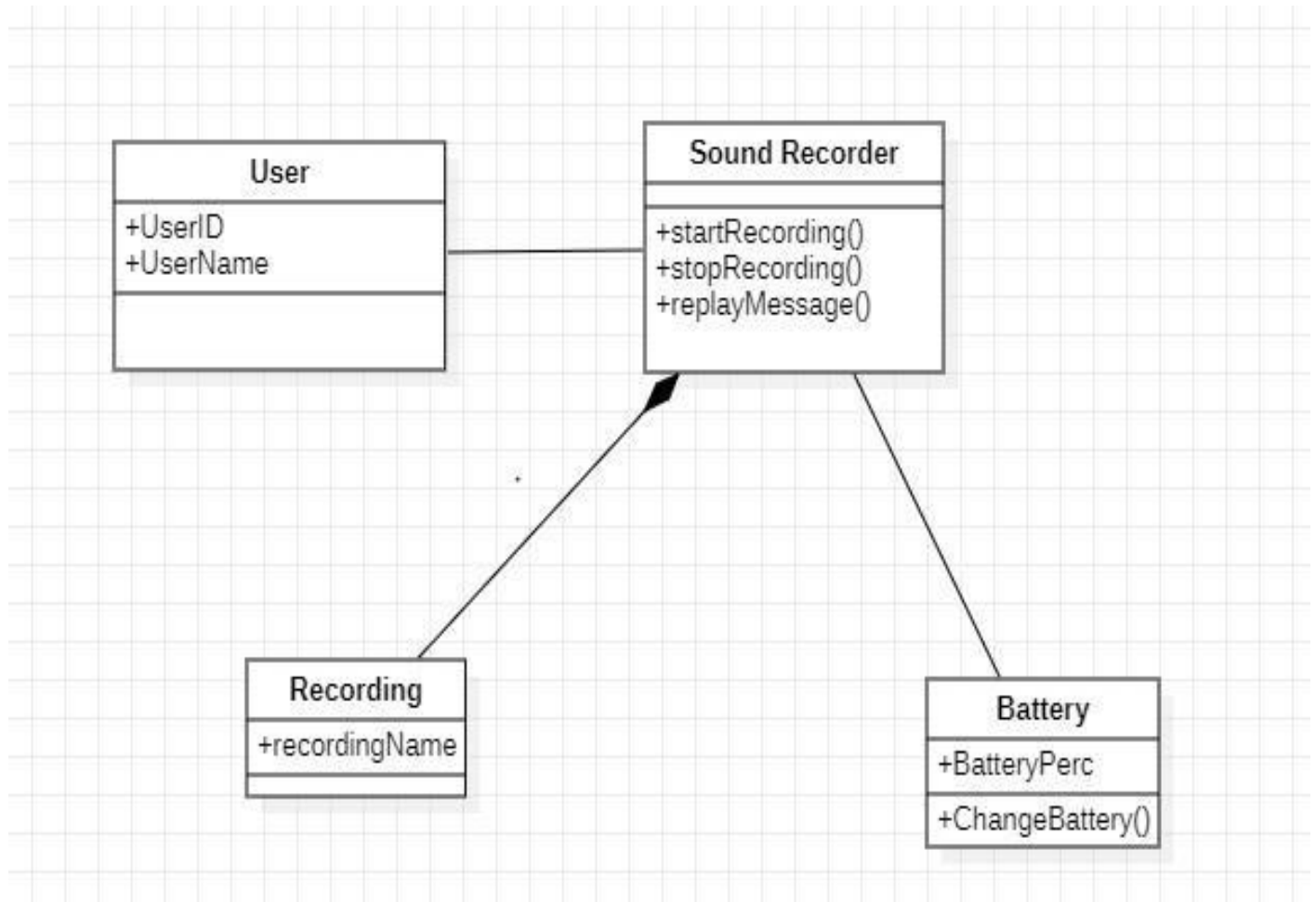
LAB-4

Use-case Diagram:



LAB-5

Class Diagram:



LAB-6

Solution:

```
public class DigitalSoundRecorder {
    private Message[] messages;
    private int batteryLevel;

    public DigitalSoundRecorder() {
        this.messages = new Message[10];
        this.batteryLevel = 100; // Assume battery level is initially full
    }

    public void recordMessage(int slot) {
        if (isBatterySufficient()) {
            // Logic to start recording a message at the specified slot
            messages[slot] = new Message();
        }
    }
}
```

```

        // Start recording process...
    } else {
        System.out.println("Battery level too low to record.");
    }
}

public void stopRecording(int slot) {
    // Logic to stop recording at the specified slot
    // Stop recording process...
}

public void replayMessage(int slot) {
    if (isBatterySufficient()) {
        // Logic to replay the message at the specified slot
        Message message = messages[slot];
        // Play message...
    } else {
        System.out.println("Battery level too low to replay message.");
    }
}

public void lockMessage(int slot) {
    messages[slot].setLocked(true);
}

public void unlockMessage(int slot) {
    messages[slot].setLocked(false);
}

private boolean isBatterySufficient() {
    return batteryLevel > 20; // Assume minimum battery level required
}

}

public class Message {
    private String content;
    private int duration;
    private boolean locked;

    public Message() {
        this.content = "";
        this.duration = 0;
        this.locked = false;
    }

    public Message(String content, int duration) {
        this.content = content;
        this.duration = duration;
        this.locked = false;
    }

    public String getContent() {
        return content;
    }
}

```

```

public int getDuration() {
    return duration;
}

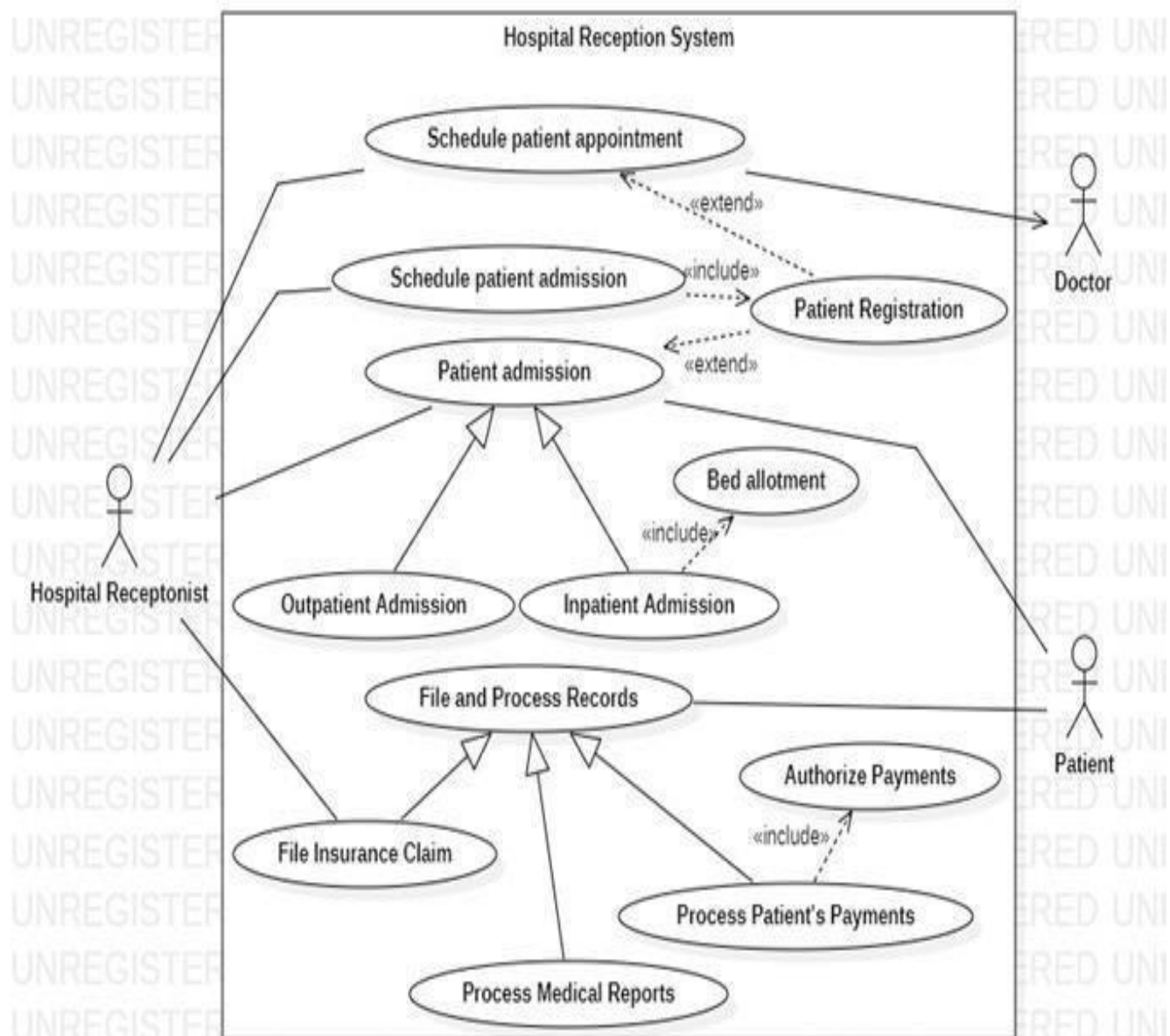
public boolean isLocked() {
    return locked;
}

public void setLocked(boolean locked) {
    this.locked = locked;
}
}

```

LAB-7

a) Use-case Diagram:



b) Use Case Descriptions :

1. Schedule Appointment:

- Description: The receptionist schedules appointments for patients based on doctor availability and patient preferences.
- Actors: Receptionist, Patient
- Flow:
 1. Receptionist checks doctor availability.
 2. Receptionist selects suitable date and time for appointment.
 3. Receptionist confirms appointment with patient.

2. Admit Patient:

- Description: The receptionist admits patients to the hospital, allocates beds in wards for inpatients, and collects necessary information.
- Actors: Receptionist, Patient
- Flow:
 1. Receptionist collects patient's personal and medical information.
 2. For inpatients, receptionist allocates a bed in a suitable ward.
 3. Receptionist updates patient records and issues an admission ID.

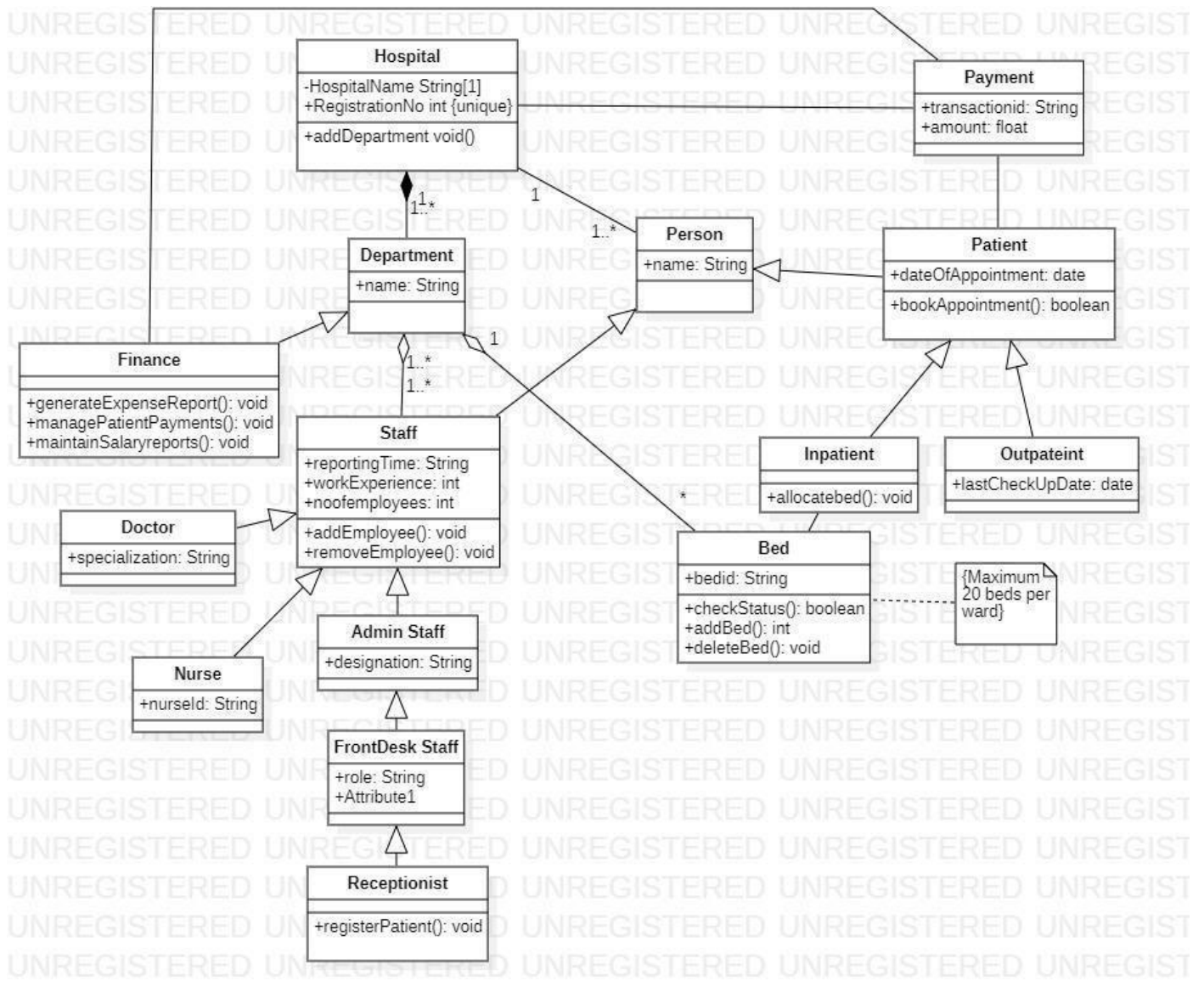
3. Process Payments:

- Description: The receptionist receives and processes patient payments for services received.
- Actors: Receptionist, Patient
- Flow:
 1. Receptionist calculates the total amount to be paid based on services availed.
 2. Receptionist accepts payment via cash, card, or other means.
 3. Receptionist issues a receipt for the payment made.

4. File Insurance Claims:

- Description: The receptionist files insurance claims on behalf of patients for reimbursement of medical expenses.
- Actors: Receptionist, Insurance Company
- Flow:
 1. Receptionist gathers necessary medical records and billing details.
 2. Receptionist submits insurance claims electronically or via mail.
 3. Receptionist tracks and follows up on insurance claim status.

c) Class Diagram :



d) Java Stub Code:

Hospital.java

```
import java.util.*;
```

```
public class Hospital {
    private String hospitalName;
    private int registrationNo;

    public Hospital(String hospitalName, int registrationNo) {
        this.hospitalName = hospitalName;
        this.registrationNo = registrationNo;
    }

    public void addDepartment() {
        // Implementation
    }
}
```

```
}  
}
```

Department.java

```
import java.util.*;  
  
public class Department {  
    public String name;  
    public Department(String name) {  
        this.name = name;  
    }  
}
```

Staff.java

```
import java.util.*;  
  
public class Staff extends Person {  
    private String reportingTime;  
    private int workExperience;  
    private int numOfEmployees;  
    public Staff(String name, String reportingTime, int workExperience, int numOfEmployees) {  
        super(name);  
        this.reportingTime = reportingTime;  
        this.workExperience = workExperience;  
        this.numOfEmployees = numOfEmployees;  
    }  
    public void addEmployee() {  
        // Implementation  
    }  
    public void removeEmployee() {  
        // Implementation  
    }  
}
```

Person.java

```
import java.util.*;  
public class Person {  
    protected String name;  
    public Person(String name) {  
        this.name = name;  
    }  
}
```

Patient.java

```
import java.util.*;  
  
public class Patient extends Person {
```

```

private Date dateOfAppointment;
public Patient(String name, Date dateOfAppointment) {
    super(name);
    this.dateOfAppointment = dateOfAppointment;
}
public boolean bookAppointment() {
    // Implementation
    return false;
}
}

```

Inpatient.java

```
import java.util.*;
```

```

public class Inpatient extends Patient {
    public Inpatient(String name, Date dateOfAppointment) {
        super(name, dateOfAppointment);
    }
    public void allocateBed() {
        // Implementation
    }
}

```

Outpatient.java

```
import java.util.*;
```

```

public class Outpatient extends Patient {
    private Date lastCheckUpDate;
    public Outpatient(String name, Date dateOfAppointment, Date lastCheckUpDate) {
        super(name, dateOfAppointment);
        this.lastCheckUpDate = lastCheckUpDate;
    }
}

```

Doctor.java

```
import java.util.*;
```

```

public class Doctor extends Staff {
    private String specialization;
    public Doctor(String name, String reportingTime, int workExperience, int numOfWorkExperience,
String specialization) {
        super(name, reportingTime, workExperience, numOfWorkExperience);
        this.specialization = specialization;
    }
}

```

Nurse.java

```
import java.util.*;

public class Nurse extends Staff {
    private String nurseld;
    public Nurse(String name, String reportingTime, int workExperience, int numOfEmployees,
String nurseld) {
        super(name, reportingTime, workExperience, numOfEmployees);
        this.nurseld = nurseld;
    }
}
```

AdminStaff.java

```
import java.util.*;

public class AdminStaff extends Staff {
    private String designation;
    public AdminStaff(String name, String reportingTime, int workExperience, int
numOfEmployees, String designation) {
        super(name, reportingTime, workExperience, numOfEmployees);
        this.designation = designation;
    }
}
```

FrontDeskStaff.java

```
import java.util.*;

public class FrontDeskStaff extends AdminStaff {
    private String role;
    private String attribute1;

    public FrontDeskStaff(String name, String reportingTime, int workExperience, int
numOfEmployees, String designation, String role, String attribute1) {
        super(name, reportingTime, workExperience, numOfEmployees, designation);
        this.role = role;
        this.attribute1 = attribute1;
    }
}
```

Receptionist.java

```
import java.util.*;

public class Receptionist extends FrontDeskStaff {
```



```

    public Receptionist(String name, String reportingTime, int workExperience, int
numOfEmployees, String designation, String role, String attribute1) {
        super(name, reportingTime, workExperience, numOfEmployees, designation, role,
attribute1);
    }
    public void registerPatient() {
        // Implementation
    }
}

```

Bed.java

```

import java.util.*;

public class Bed {
    private String bedId;
    private boolean occupied;

    public Bed(String bedId) {
        this.bedId = bedId;
        this.occupied = false;
    }

    public boolean checkStatus() {
        return occupied;
    }
    public void setOccupied(boolean occupied) {
        this.occupied = occupied;
    }
}

```

Finance.java

```

import java.util.*;

public class Finance extends Staff {
    public Finance(String name, String reportingTime, int workExperience, int numOfEmployees)
{
        super(name, reportingTime, workExperience, numOfEmployees);
    }
    public void generateExpenseReport() {
        // Implementation
    }
    public void managePatientPayments() {
        // Implementation
    }
    public void maintainSalaryReports() {
        // Implementation
    }
}

```

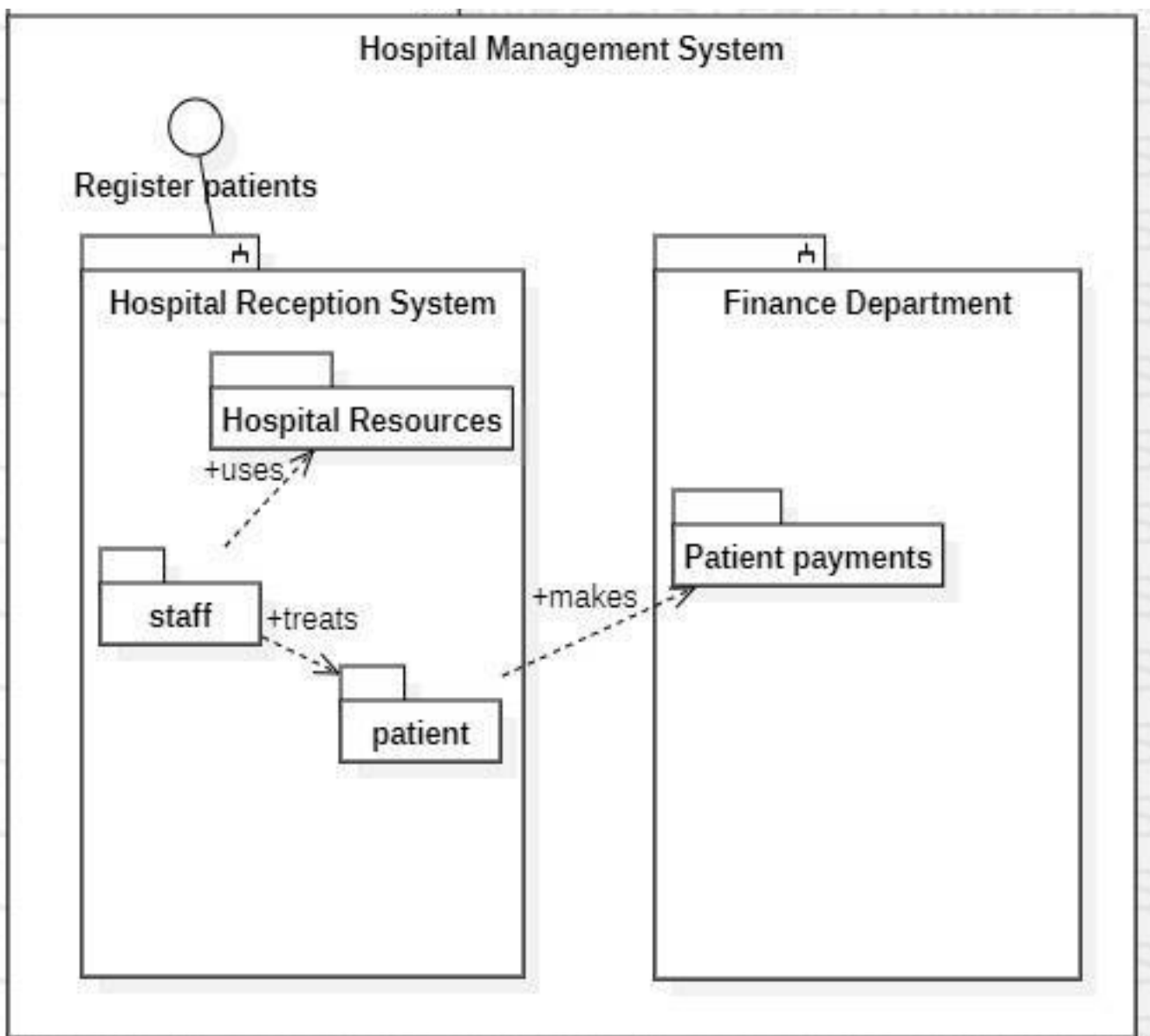
```
}
```

Payment.java

```
import java.util.*;
```

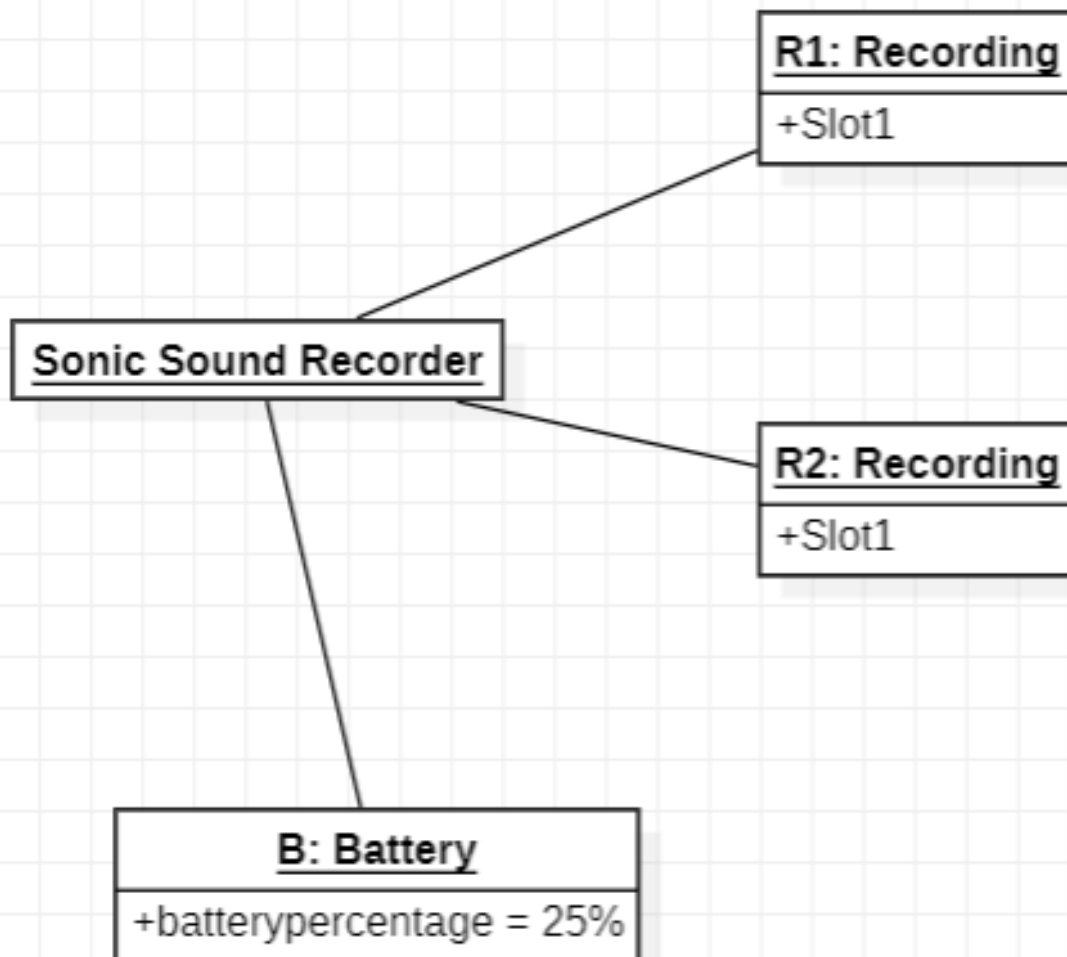
```
public class Payment {  
    private String transactionId;  
    private float amount;  
  
    public Payment(String transactionId, float amount) {  
        this.transactionId = transactionId;  
        this.amount = amount;  
    }  
}
```

e) Object Diagram:



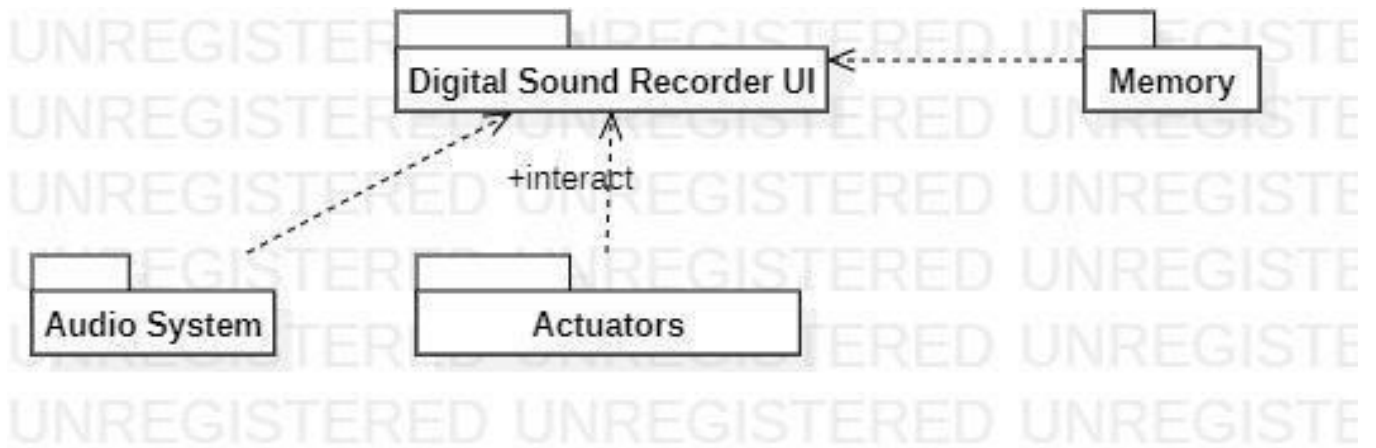
LAB-8

Object Diagram:



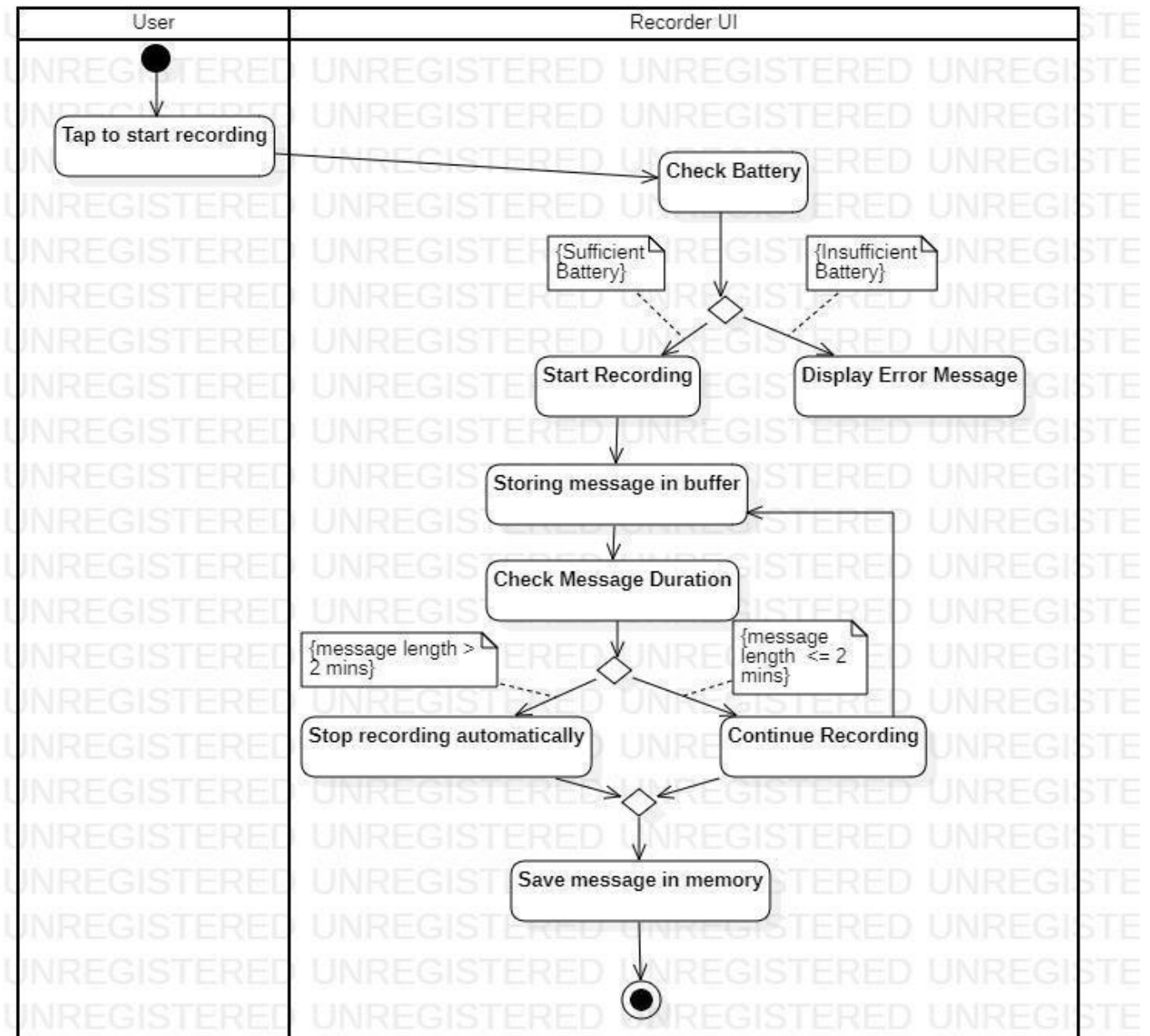
LAB-9

Package Diagram:



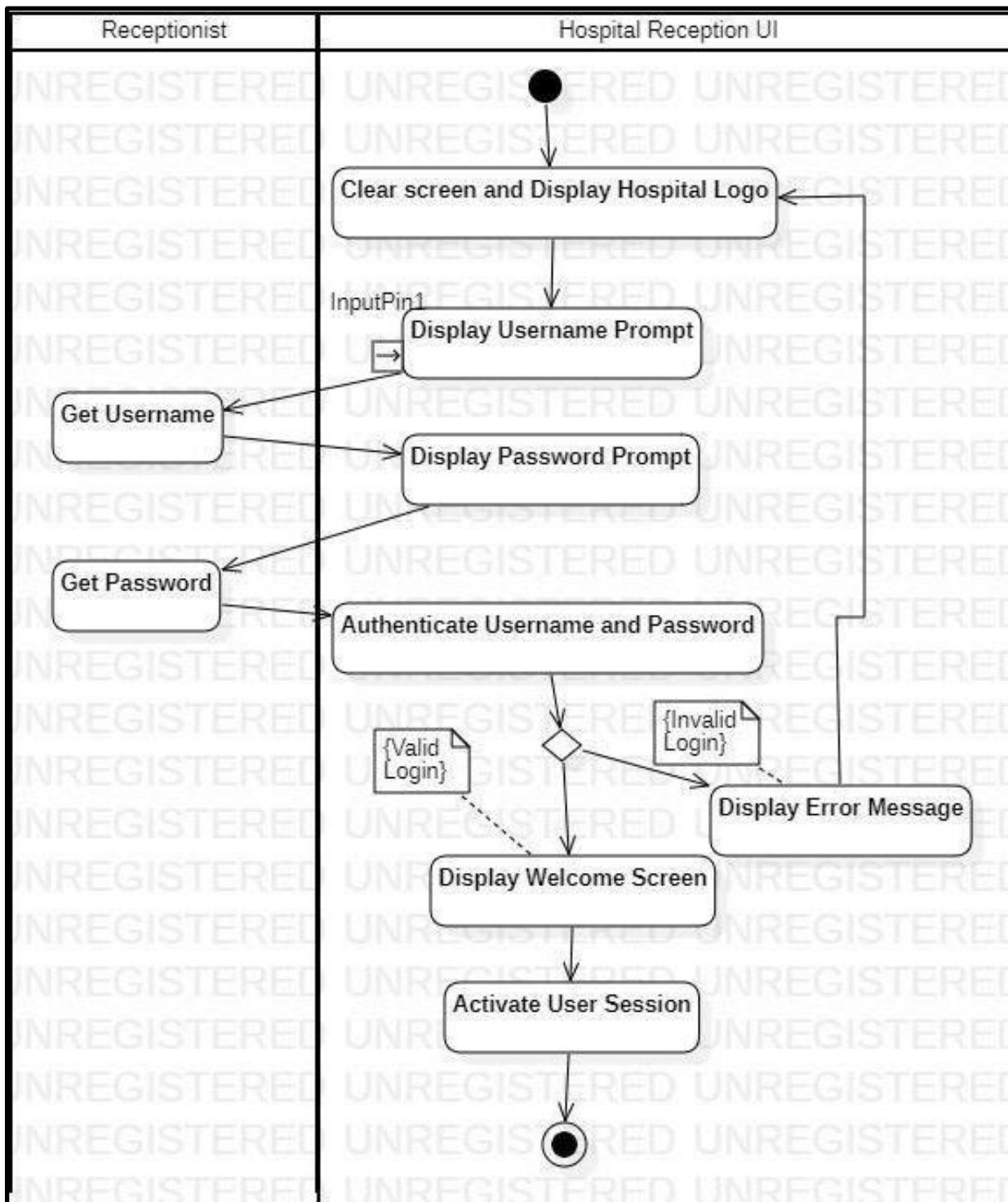
LAB-10

Activity Diagram:



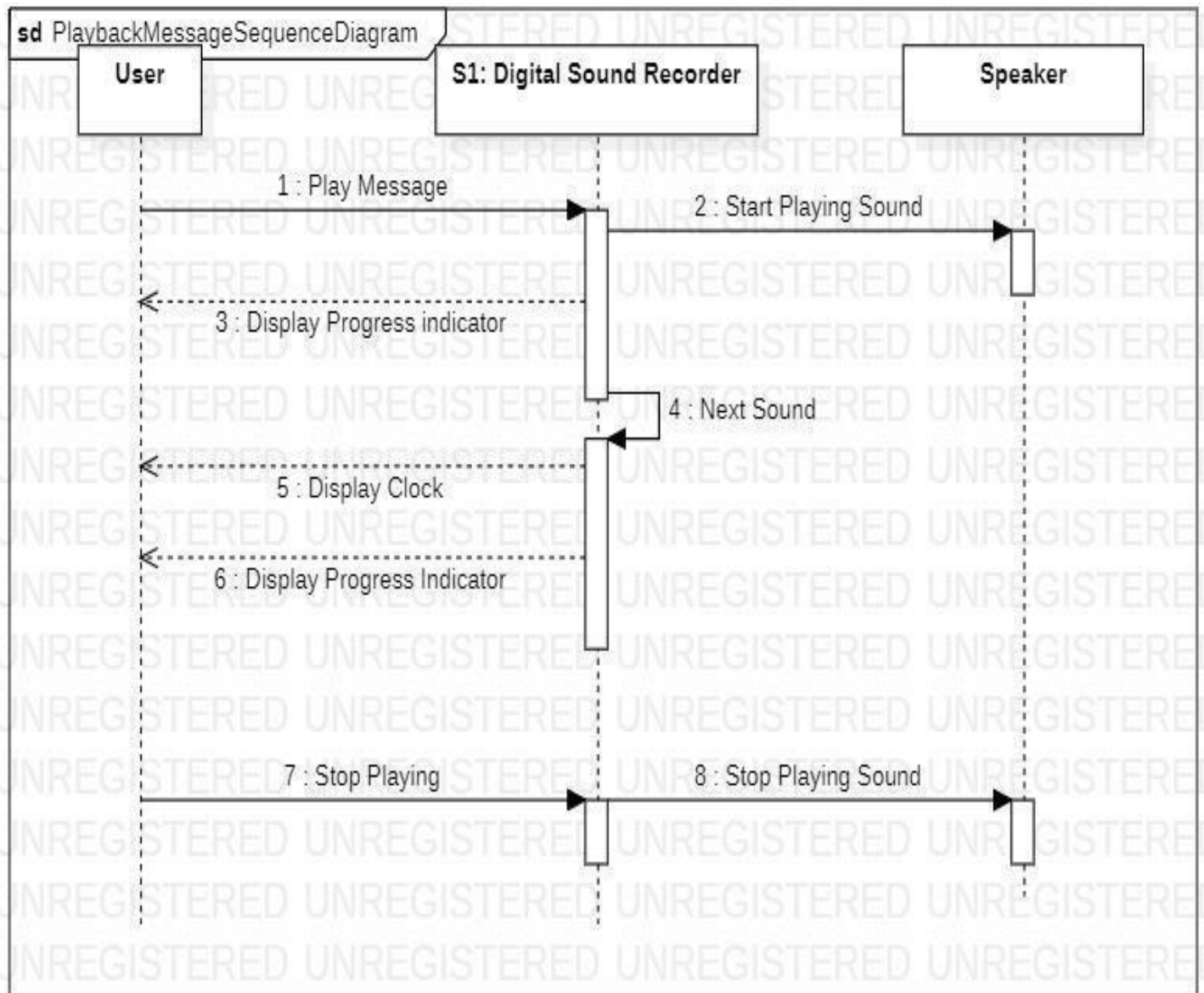
LAB-11

Activity Diagram:



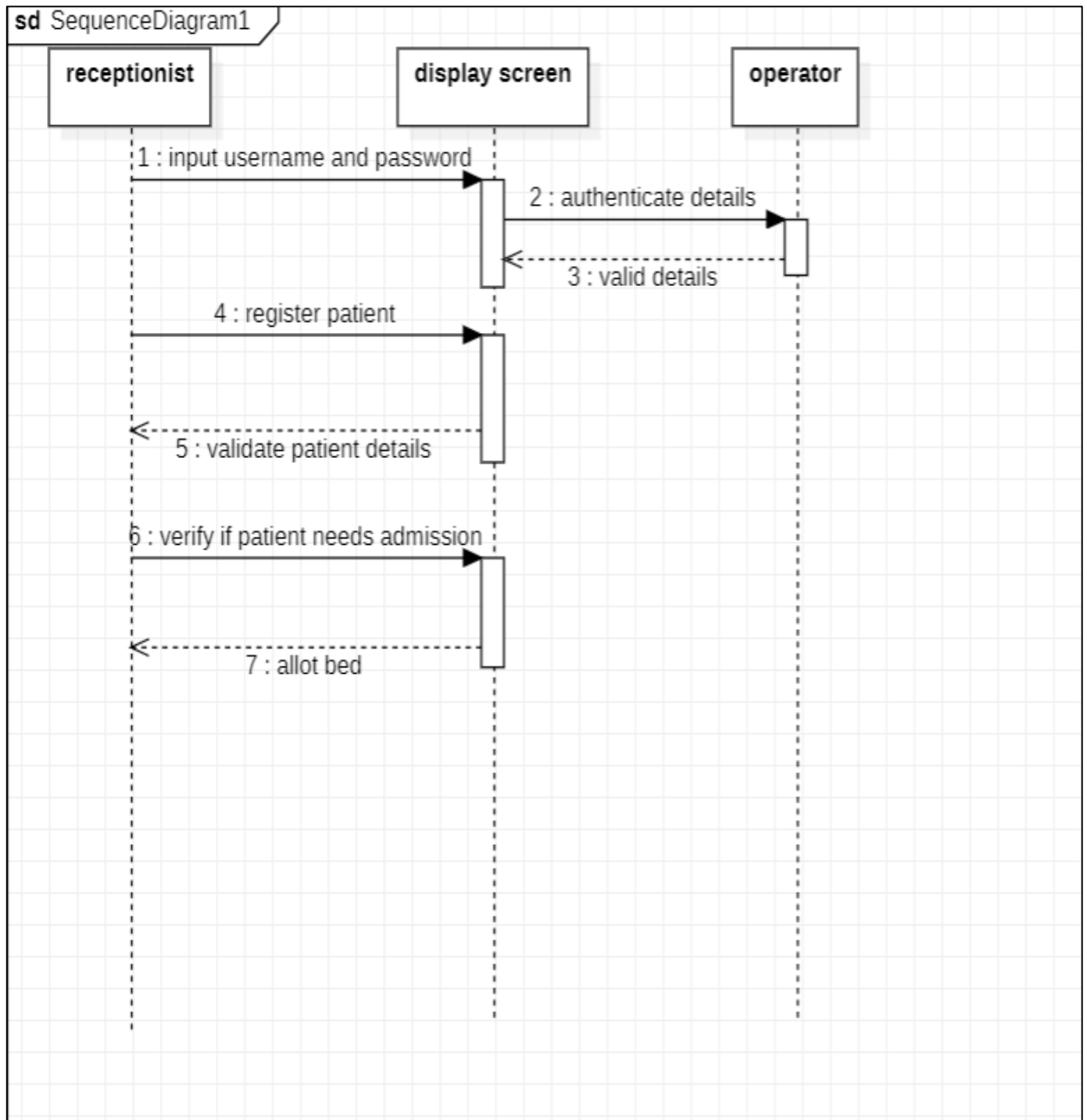
LAB-12

Sequence Diagram:



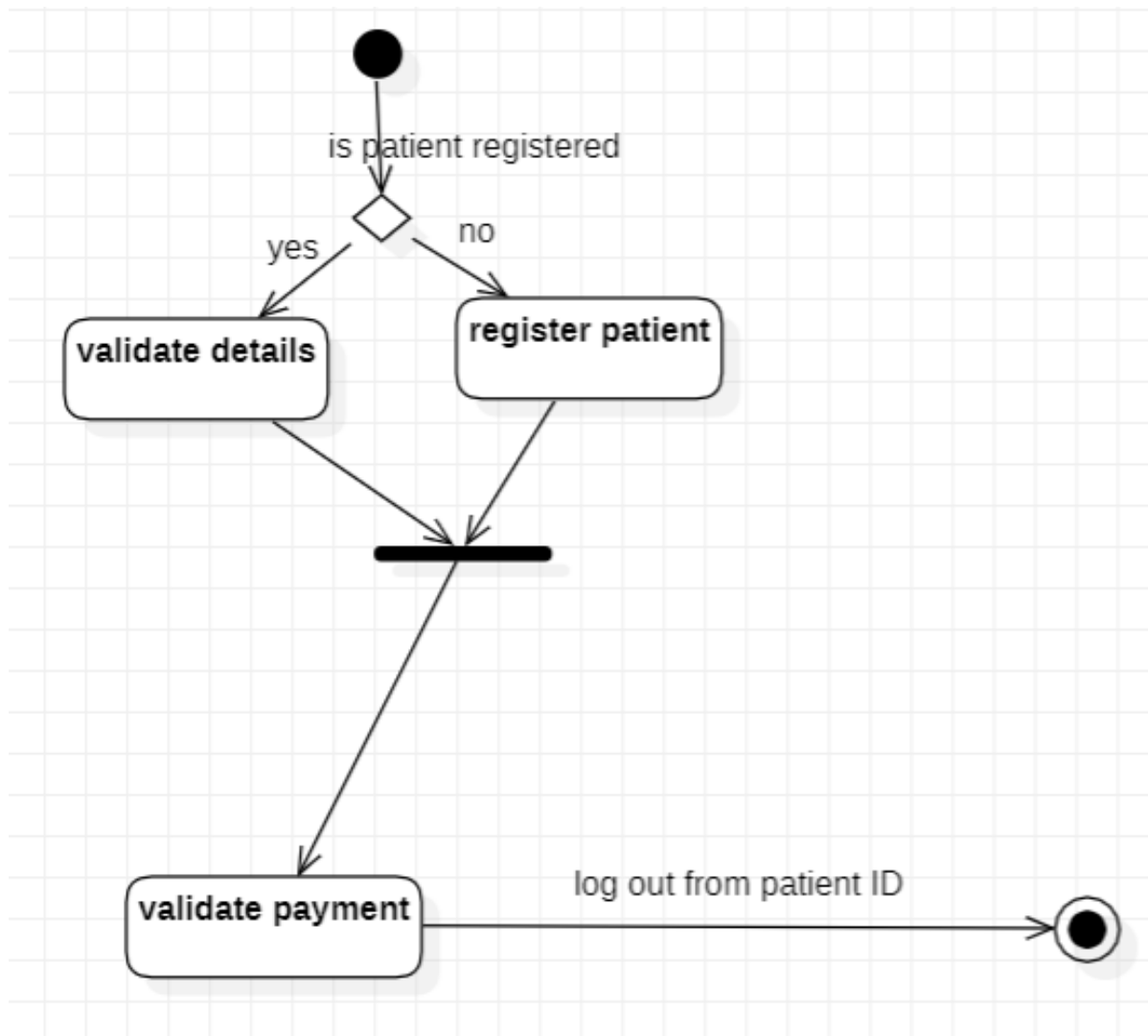
LAB-13

Sequence Diagram:



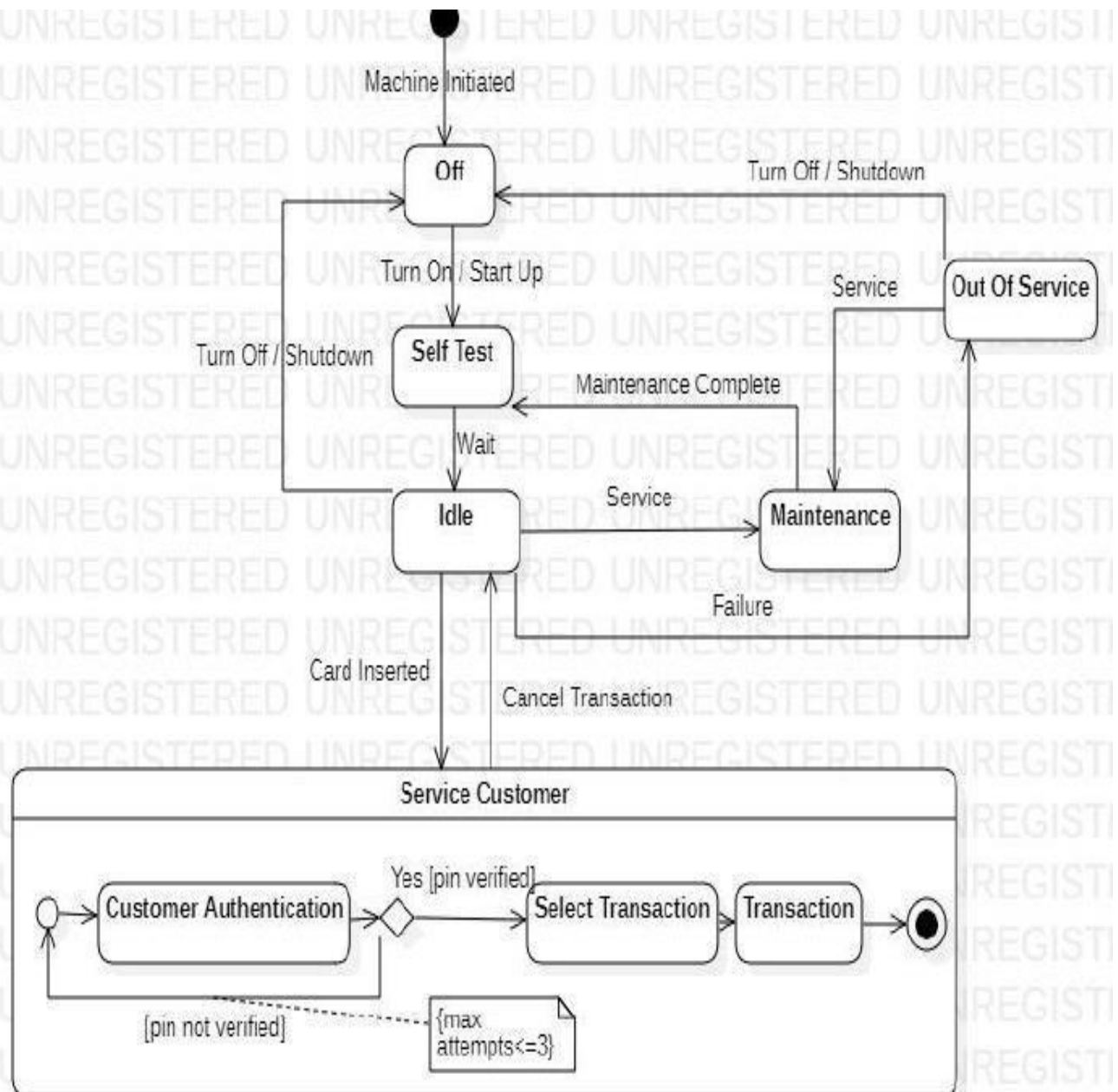
LAB-14

State Diagram:



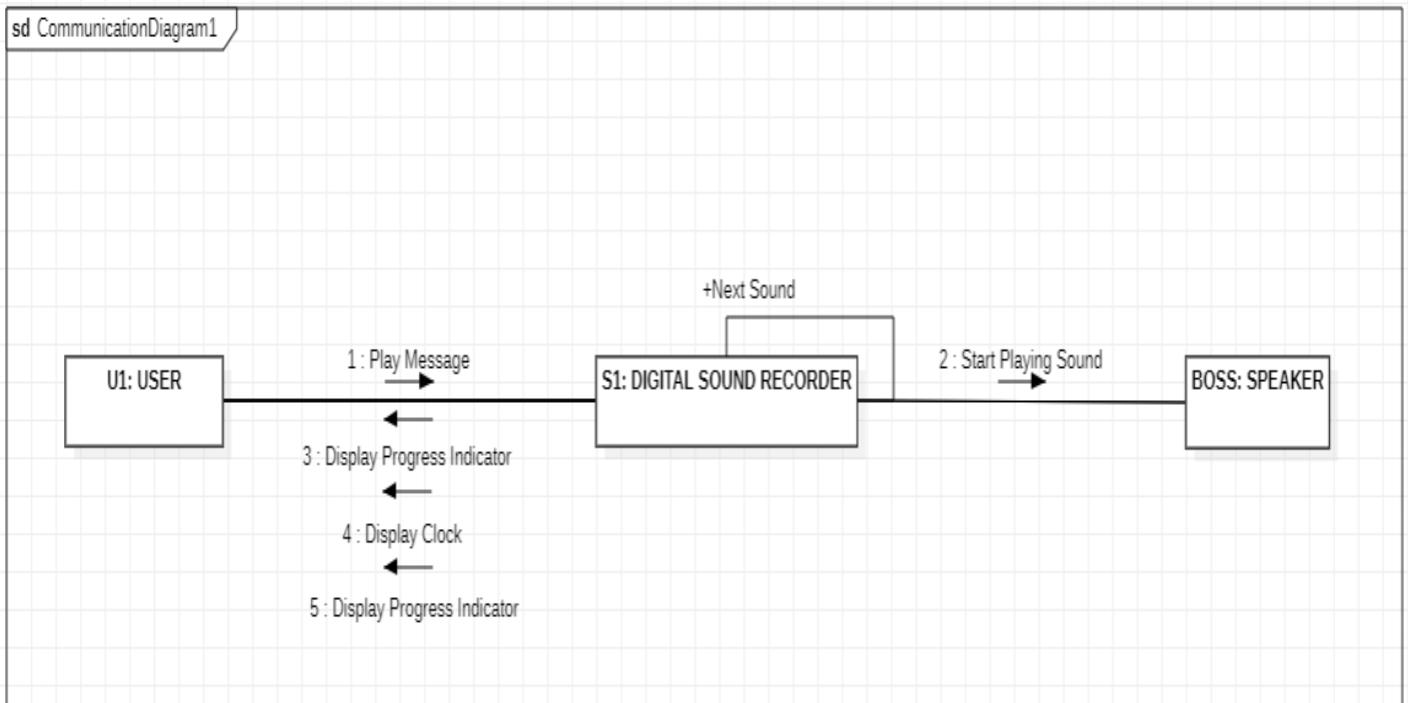
LAB-15

State Diagram:



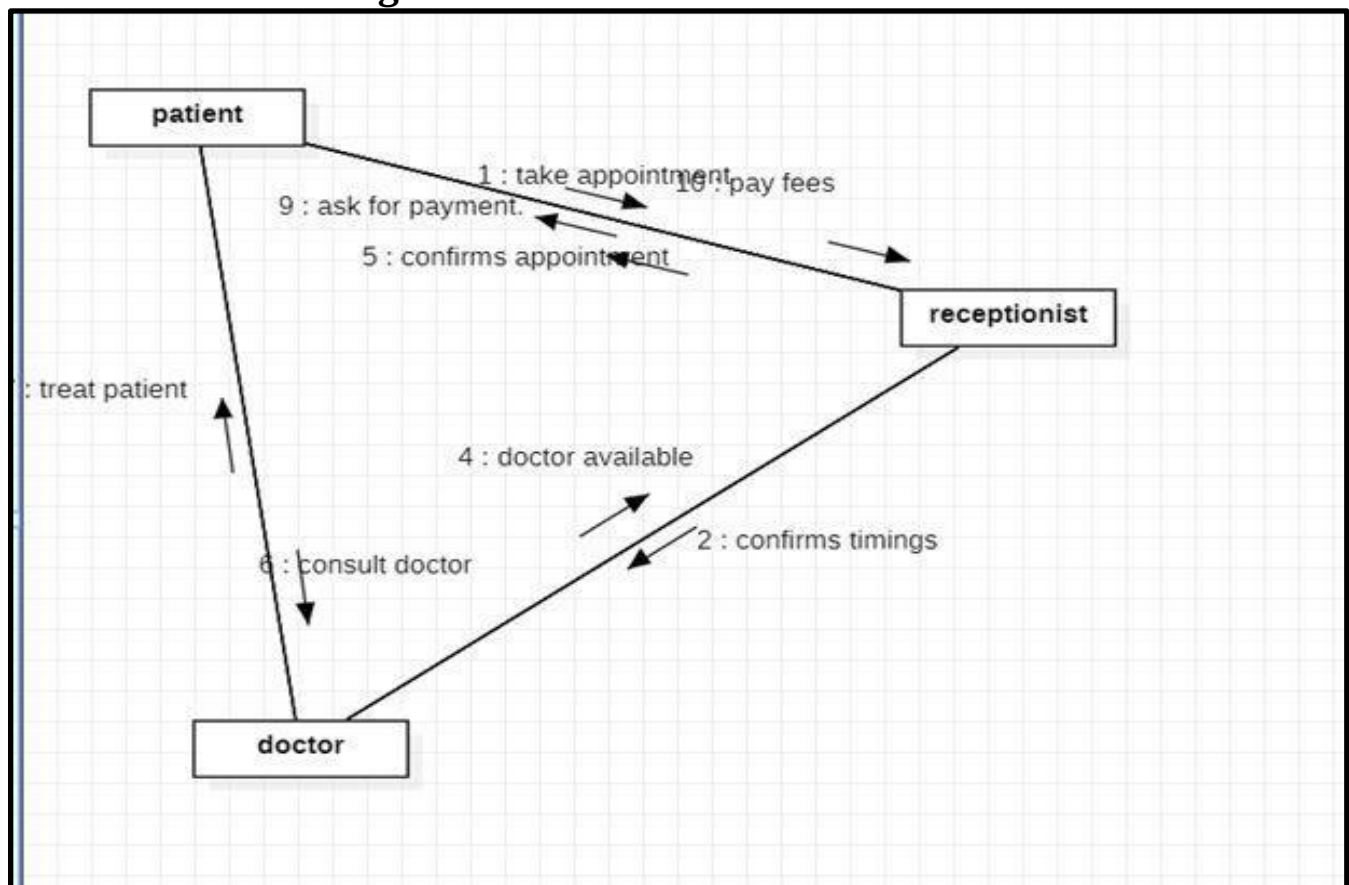
LAB-16

Communication Diagram:



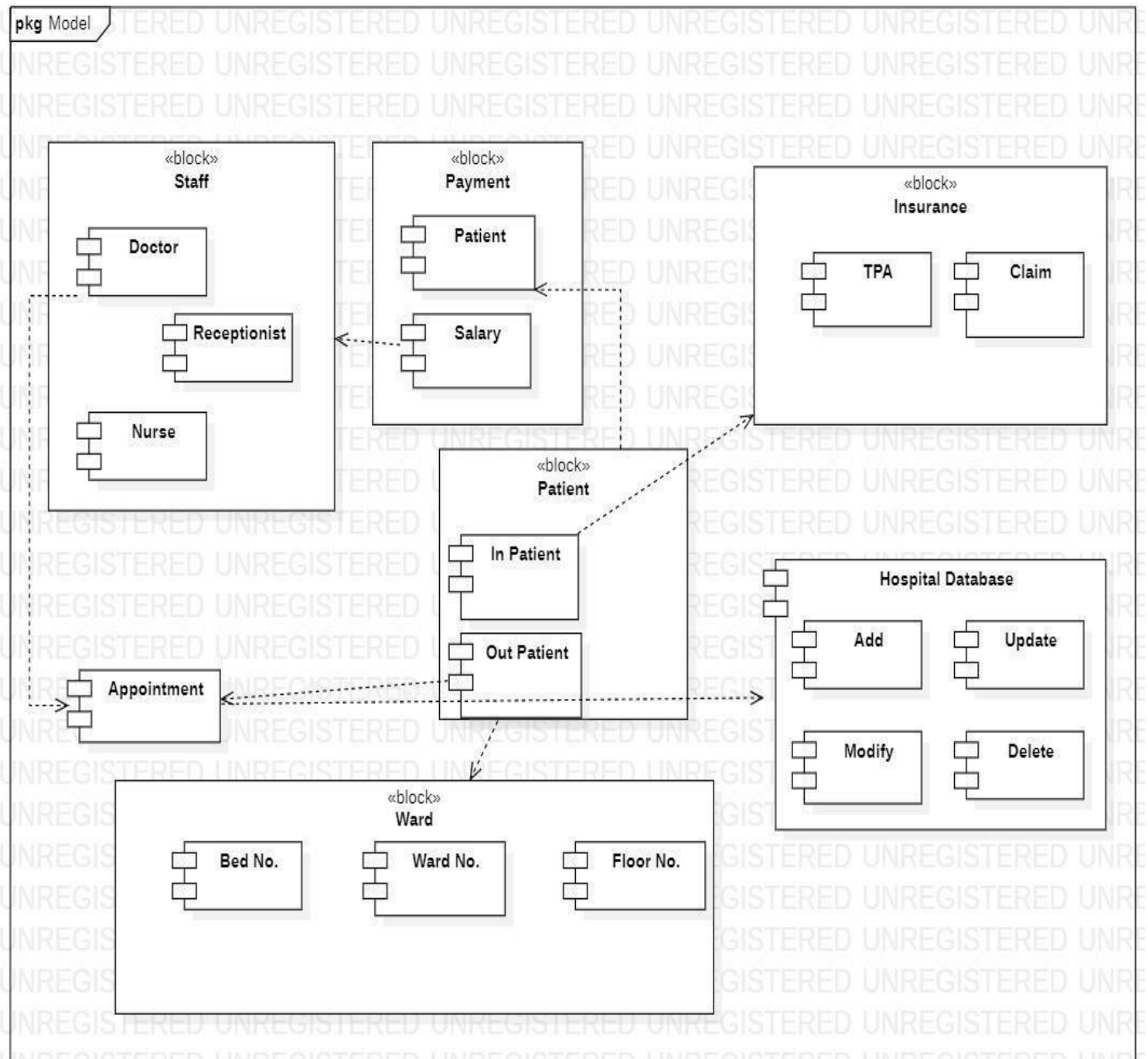
LAB-17

Communication Diagram:



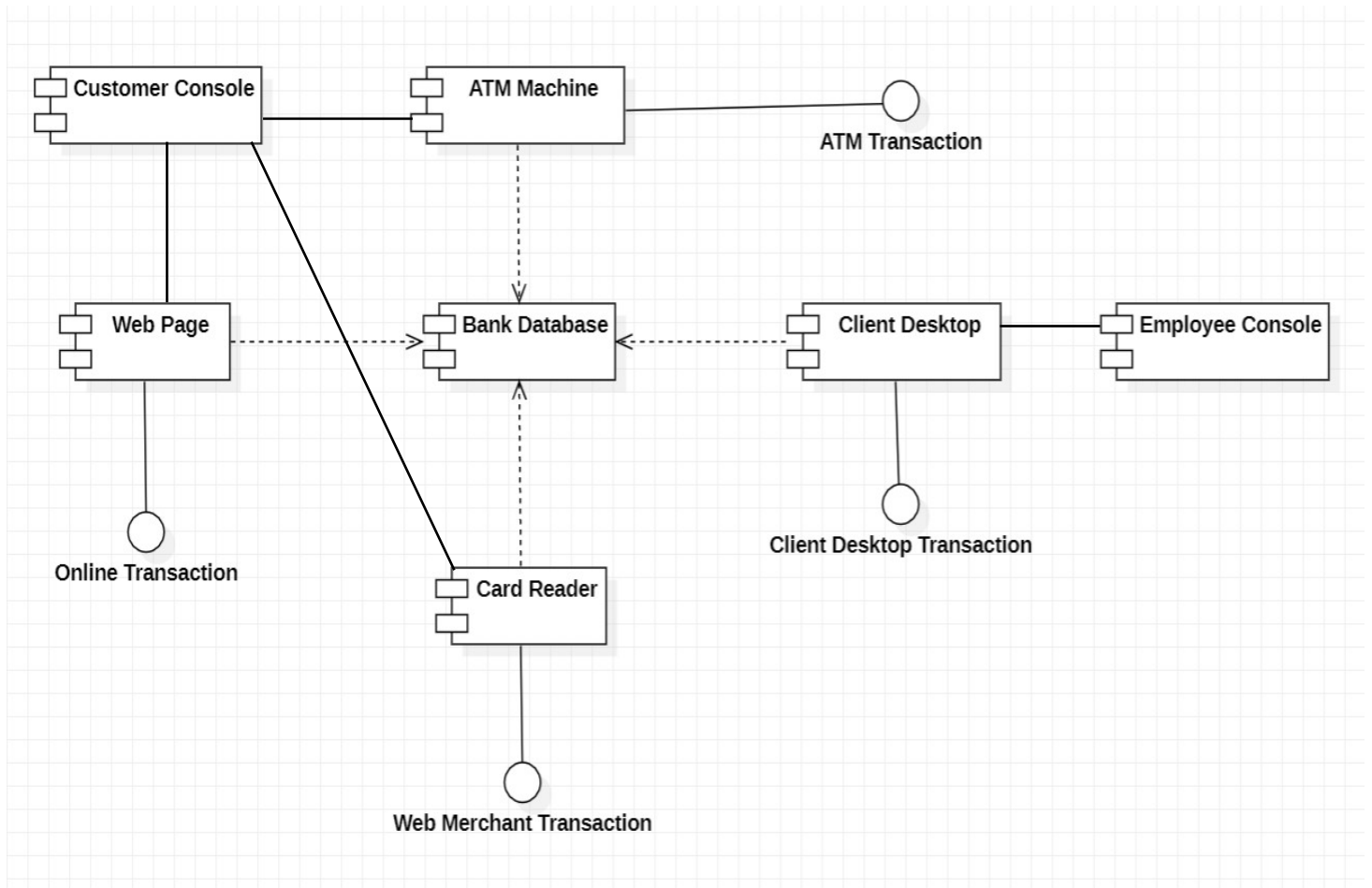
LAB-18

Component Diagram:



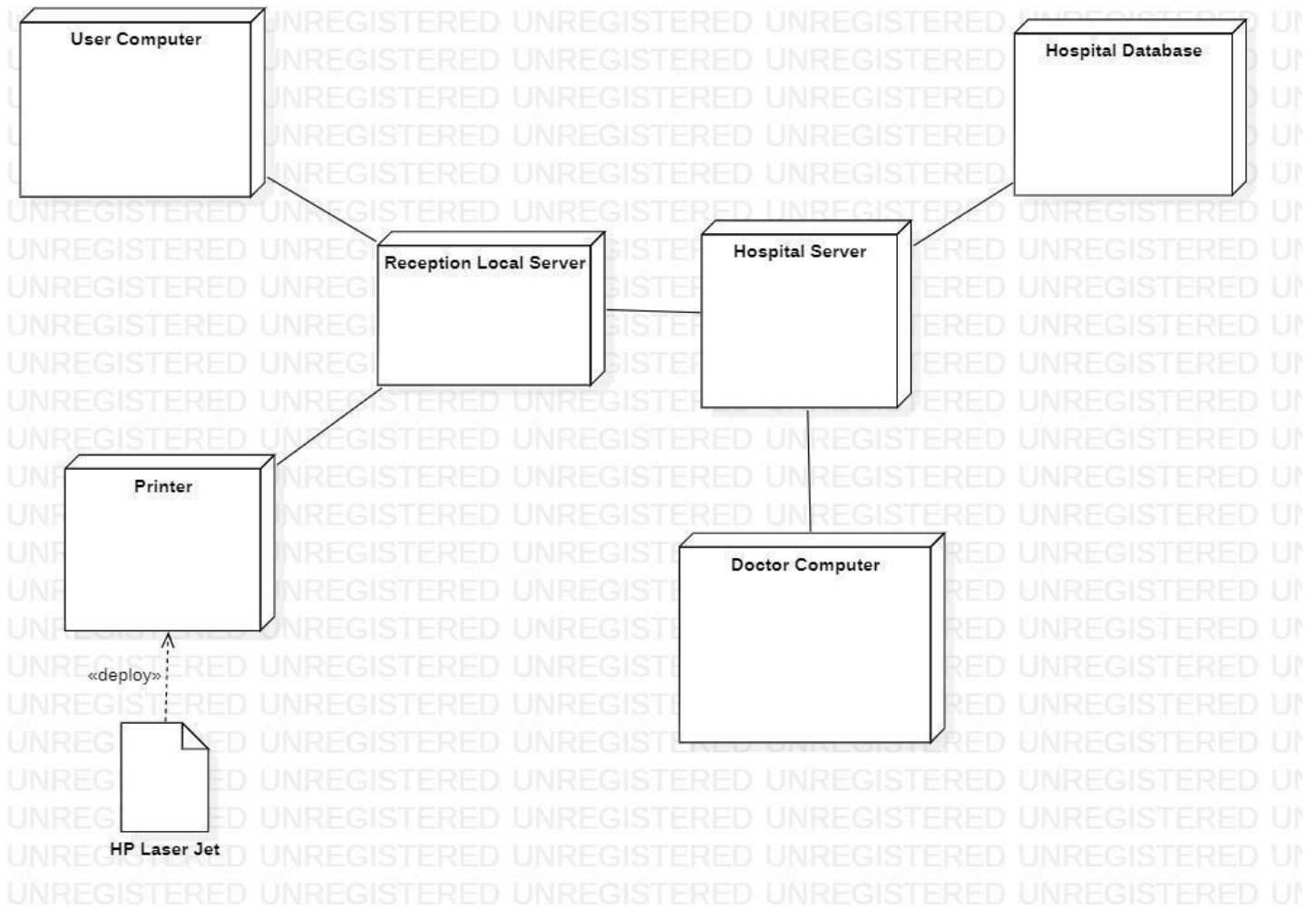
LAB-19

Component Diagram:



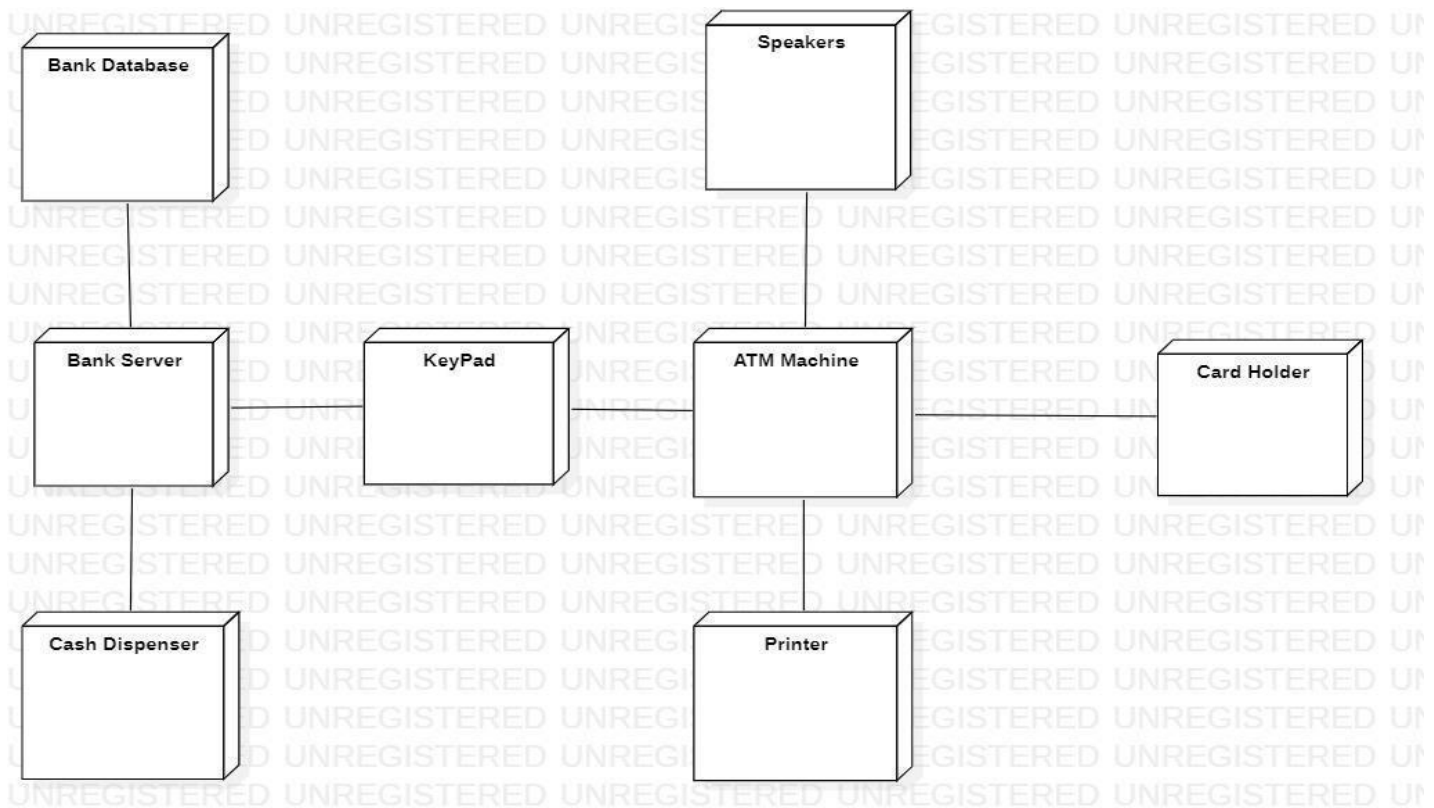
LAB-20

Deployment Diagram:



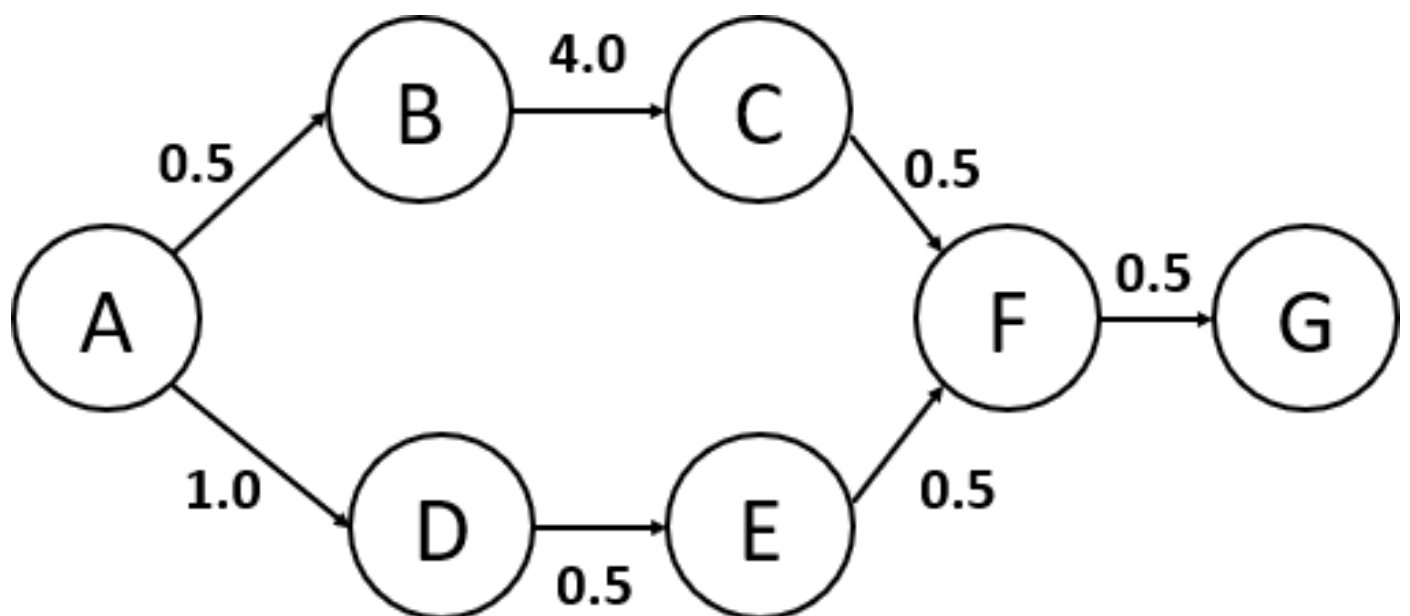
LAB-21

Deployment Diagram:



LAB-22

Diagram:



LAB-23

Gantt Chart:

PARTY PLAN GANTT CHART

